

Operating Systems Mastery

From process basics to the FAANG OS round
9 diagrams, 17 model answers, a recently-asked set, 50 FAQs

[@rathoreadiitya](#)

v1.0 · Updated 2026-06-06

Brand new to OS? Start at **§3 Start here: the student minimum** below, then read the core concepts in order.

Two readers, one sheet: a 2nd-year who has never drawn a Gantt chart, and a final-year about to face an OS round at Microsoft or a service company. Read top to bottom if OS is new. Short on time? Use the table of contents to jump.

Inside this sheet

1. Brand-new note + who this sheet is for
2. Inside this sheet (you are here)
3. Start here: the student minimum - what an OS round really tests
4. Why OS feels abstract to students
5. Process vs program vs thread
6. The PCB - what the OS remembers about a process
7. Process states + the lifecycle diagram
8. Context switch - the cost of multitasking
9. fork / exec, zombie + orphan processes
10. User mode vs kernel mode + system calls
11. Thread models (many-to-one, one-to-one, many-to-many)
12. CPU scheduling - formulas, FCFS / SJF / SRTF / RR / Priority / MLFQ + multicore bonus
13. Beyond the basics (advanced, not all fresher-required)
14. Synchronization - race condition, critical section, mutex, semaphore, monitor
15. Classic synchronization problems
16. Deadlock - 4 conditions, prevention, Banker's, vs starvation vs livelock
17. Memory management - fragmentation, paging, segmentation, virtual memory
18. Page replacement - FIFO, Optimal, LRU, Clock + Belady's anomaly
19. Thrashing, working set, TLB
20. Disk scheduling - FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK
21. File system allocation (brief)
22. Comparison tables (the ones interviewers love)
23. Interview triggers - if they ask X, say Y
24. Common mistakes + traps
25. Model interview answers - say it like this (17 questions)
26. Recently asked in interviews - what current cycles hit
27. Interview FAQ bank - 50 questions
28. The 30-minute revision card
29. Final word

If you have only 30 minutes - jump to §28 Revision card.

3. Start here: the student minimum

An OS round for a fresher is not the whole textbook. It is a small core asked over and over. Learn this core cold and you will not drop easy points in most rounds.

The five must-knows:

1. Process vs thread - what they share, what they don't, and why threads are cheaper to switch.
2. CPU scheduling - draw a Gantt chart and compute average waiting time and turnaround time by hand for FCFS, SJF, SRTF, and Round Robin.
3. Deadlock - the four Coffman conditions, and that ALL four must hold at once.
4. Paging + page replacement - how a logical address becomes physical, and FIFO vs LRU vs Optimal (plus Belady's anomaly).
5. Synchronization basics - race condition, critical section, mutex vs semaphore.

The one hands-on thing to drill: take any process table (arrival time + burst time), draw the Gantt chart, and compute average waiting time and turnaround time for FCFS, SJF, SRTF, and Round Robin. If you can do that without notes, you are ahead of most candidates.

Where to jump: scheduling is §12, deadlock is §16, paging and page replacement are §17 and §18, synchronization is §14. Everything before §13 is the fresher core. Everything after is depth that separates a good answer from a great one.

4. Why OS feels abstract to students

DSA has visible inputs and outputs. OS is invisible: it is the referee that decides which program runs, where its memory lives, and when it has to wait. You never see it work, so it feels like trivia. It stops feeling abstract the moment you map each idea to one everyday scene - a mess token queue, a hostel bathroom, two people stuck in a doorway. That is what the Aha boxes in this sheet do.

5. Process vs program vs thread

A program is a file on disk - dead code. A process is that program loaded into memory and running, with its own address space, registers, and state. A thread is a single line of execution inside a process. One process can hold many threads that share its memory but each keep their own stack and registers.

Aspect	Process	Thread
Memory	Own private address space	Shares the process address space
Creation cost	Heavy (new address space)	Light (shares memory)
Switch cost	High (swap address space + cache)	Low (same address space)
Crash impact	One process crash does not kill others	A bad thread can crash the whole process
Communication	IPC (pipes, shared memory, messages)	Shared variables (needs locks)

💡 Aha - Program ek recipe hai jo book mein likhi hai, process us recipe ko actually kitchen mein bana rahe ho, aur threads woh do-teen log hain jo ek hi kitchen mein milke same dish banate hain - bartan share karte hain par apne apne haath se kaam karte hain.

6. The PCB - what the OS remembers about a process

The Process Control Block (PCB) is the OS's record card for one process. Every context switch saves the running process's state into its PCB and loads the next one's. The PCB holds:

- Process ID (PID) and parent PID.
- Process state (new, ready, running, waiting, terminated).
- Program counter - the next instruction to run.
- CPU registers - so execution resumes exactly where it left off.
- Scheduling info - priority, queue pointers.
- Memory management info - page tables, base and limit registers.
- Accounting + I/O status - open files, allocated devices.

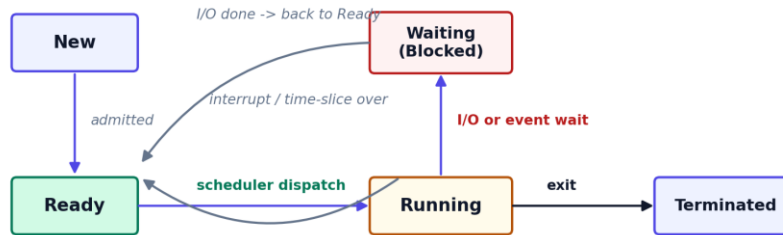
💡 Aha - PCB tumhare process ka Aadhaar card hai jo OS ke paas rehta hai - naam, state, kahan tak kaam hua, kaunsi files khuli hain, sab likha hota hai, taaki jab process ko dobara CPU mile toh OS bilkul wahin se shuru kar de jahan chhoda tha.

7. Process states + the lifecycle

A process moves through five states. The key rule freshers miss: a Running process that asks for I/O goes to Waiting, not back to Ready. It returns to Ready only when the I/O finishes.

- New - being created, not yet admitted.
- Ready - in memory, waiting only for the CPU.
- Running - currently on the CPU (one per core).
- Waiting / Blocked - waiting for I/O or an event, not the CPU.
- Terminated - finished, being cleaned up.

Process states: the journey from New to Terminated



Only ONE process is Running per CPU core at a time. The scheduler picks the next Ready process; a Running process that asks for I/O goes to Waiting, not back to Ready.

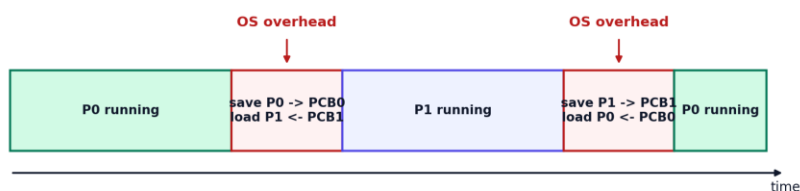
The five-state lifecycle. Dispatch = Ready to Running, time-slice over = Running back to Ready, I/O = Running to Waiting.

💡 Aha - Socho ek doctor ke clinic mein ho - Ready matlab waiting room mein baithe ho, Running matlab andar doctor ke saamne ho, aur Waiting tab hai jab doctor ne blood test likh diya aur tum lab gaye - ab tum doctor ka time nahi le rahe, report aane ka wait kar rahe ho.

8. Context switch - the cost of multitasking

A context switch is the OS saving one process's state to its PCB and loading another's. It is pure overhead: during the switch the CPU does no useful work. Too many switches (tiny time quanta, constant preemption) waste cycles and thrash the CPU cache.

Context switch: saving P0, loading P1 (pure overhead)



During the red blocks the CPU does ZERO useful work. The PCB is where each process's registers, program counter, and stack pointer are parked. Too many switches = thrashing the cache.

The red blocks are pure overhead. The CPU computes nothing while it parks one process and loads the next.

💡 Aha - Context switch waise hi hai jaise ek student do subjects ke beech switch karta hai - har baar pehli kitaab band karke bookmark lagao, doosri kholo, yaad karo kahan the. Jitni baar switch karoge, utna time sirf switch karne mein chala jaata hai, padhai mein nahi.

9. fork / exec, zombie + orphan processes

fork() creates a child process - a near-copy of the parent. It returns 0 to the child and the child's PID to the parent, so one call returns twice. exec() then replaces the child's memory image with a new program. The fork-then-exec pair is how a shell launches any command.

C - fork then exec

```
pid = fork();           // one call, returns twice
if (pid == 0) {         // child branch (got 0)
    exec("./prog");     // replace image with a new program
} else {               // parent branch (got child PID)
    wait(&status);      // reap the child, avoid a zombie
}
```

- **Zombie** - the child has finished but the parent has not called wait() to read its exit status. The PCB lingers in the table as a zombie until reaped.
- **Orphan** - the parent dies before the child. The child is re-parented to init (PID 1), which adopts and eventually reaps it.

💡 **Aha - Zombie** woh bachcha hai jiska kaam khatam ho gaya par parent ne abhi tak uska result note nahi kiya, isliye record table mein atka pada hai. **Orphan** woh hai jiska parent pehle chala gaya, toh OS ka init process use god le leta hai.

10. User mode vs kernel mode + system calls

The CPU runs in one of two modes. In user mode, application code runs with limited privileges - it cannot touch hardware or other processes' memory directly. In kernel mode, the OS runs with full access. A program crosses from user to kernel mode only through a system call (a controlled doorway), which triggers a trap into the OS.

- **System calls:** process control (fork, exec, exit), file ops (open, read, write, close), device ops, information (getpid), communication (pipe).
- A trap / software interrupt switches the CPU to kernel mode, runs the handler, then returns to user mode.
- This boundary is why a user program cannot just format your disk - it has to ask the kernel, which checks permissions.

💡 **Aha - User mode vs kernel mode** bank jaisa hai - tum customer ho, cash counter ke peeche nahi ja sakte. Paisa chahiye toh form bharke (system call) cashier (kernel) ko dete ho, woh check karke kaam karta hai. Seedha locker mein haath nahi daal sakte.

11. Thread models

User threads are managed by a library in user space. Kernel threads are managed by the OS. How user threads map to kernel threads gives three models:

Model	Mapping	Trade-off
Many-to-one	Many user threads -> 1 kernel thread	One blocking call blocks ALL threads, no true parallelism
One-to-one	Each user thread -> its own kernel thread	True parallelism, more overhead per thread (Windows, Linux)
Many-to-many	Many user threads -> a pool of kernel threads	Balances parallelism and cost, more complex to build

💡 **Aha - Many-to-one matlab ek hi cashier ke through पूरी line - ek customer ka kaam atka toh sab atak gaye. One-to-one matlab har customer ka apna cashier - tez par mehnga. Many-to-many matlab kuch cashiers ko line smartly baant do.**

12. CPU scheduling

When several processes are Ready, the scheduler picks who runs next. This is the single most-asked OS topic, because it is the one place you compute numbers by hand. First, the three formulas you will use in every Gantt question:

Turnaround Time = Completion Time - Arrival Time. Waiting Time = Turnaround Time - Burst Time. Response Time = first time on CPU - Arrival Time.

Scheduling criteria interviewers ask you to optimize:

- CPU utilization - keep the CPU busy (higher is better).
- Throughput - processes completed per unit time (higher is better).
- Turnaround time - total time from arrival to completion (lower is better).
- Waiting time - time spent in the ready queue (lower is better).
- Response time - arrival to first CPU response (lower is better for interactive systems).

Preemptive vs non-preemptive:

- Non-preemptive - once a process gets the CPU it runs until it finishes or blocks (FCFS, SJF).
- Preemptive - the OS can take the CPU away mid-run for a higher-priority or shorter job (SRTF, Round Robin, preemptive Priority).

💡 **Aha - Scheduler ek traffic signal hai - sab gaadiyan (processes) ready hain par ek time pe ek hi lane ko green milta hai. Algorithm decide karta hai green kise mile: jo pehle aaya (FCFS), jiska kaam chhota hai (SJF), ya har lane ko baari baari thoda time (Round Robin).**

We use one process set for all four worked examples so you can see the same jobs scheduled differently:

Process	Arrival Time	Burst Time
P1	0	7
P2	2	4
P3	4	1
P4	5	4

Each table below drops the AT and BT input columns (they are in the process set above) and keeps only the computed CT, TAT, and WT. Formulas are in the amber box at the top of this section.

FCFS - First Come First Served (non-preemptive)

Run jobs in arrival order. One long job at the front makes everyone behind it wait - the convoy effect.

Gantt: P1[0-7] P2[7-11] P3[11-12] P4[12-16]

P	CT	TAT (CT-AT)	WT (TAT-BT)
P1	7	7	0
P2	11	9	5
P3	12	8	7
P4	16	11	7

FCFS: Average TAT = $35/4 = 8.75$ Average WT = $19/4 = 4.75$

💡 Aha - FCFS bilkul mess ki line hai - jo pehle aaya use pehle khana milega, chahe peeche wale ko sirf ek roti chahiye. Aage wale bhaiya ki poori thali ban-ne tak sabko khade rehna padta hai. Yahi convoy effect hai.

SJF - Shortest Job First (non-preemptive)

Pick the arrived job with the smallest burst time. Gives the minimum possible average waiting time, so it is a theoretical optimum (you must know burst times in advance). Long jobs can starve.

Gantt: P1[0-7] P3[7-8] P2[8-12] P4[12-16]

P	CT	TAT (CT-AT)	WT (TAT-BT)
P1	7	7	0
P3	8	4	3
P2	12	10	6
P4	16	11	7

SJF: Average TAT = $32/4 = 8.0$ Average WT = $16/4 = 4.0$

💡 Aha - SJF wala mess manager smart hai - jiski thali sabse chhoti hai use pehle nipta deta hai taaki overall sabka wait kam ho. Dikkat: agar tumhari badi thali hai aur chhoti thaliyan aati rahein, toh tum bhooke khade reh jaoge. Yahi starvation hai.

SRTF - Shortest Remaining Time First (preemptive SJF)

Preemptive SJF: at each arrival, if a new job has a shorter remaining time than the running one, switch. Lowest average waiting time of the four, but the most context switches.

Gantt: P1[0-2] P2[2-4] P3[4-5] P2[5-7] P4[7-11] P1[11-16]

P	CT	TAT (CT-AT)	WT (TAT-BT)
P1	16	16	9
P2	7	5	1
P3	5	1	0
P4	11	6	2

SRTF: Average TAT = $28/4 = 7.0$ Average WT = $12/4 = 3.0$

💡 Aha - SRTF mess manager aur bhi besabra hai - beech mein agar koi chhoti thali wala aa jaaye toh woh chal rahe bade order ko rok ke pehle chhote ko nipta deta hai. Sabka average wait sabse kam, par bade order wale ko baar baar roka jaata hai.

Round Robin (preemptive, time quantum)

Give each Ready process a fixed quantum, then cycle. Nobody starves and response time is great. Too small a quantum wastes time switching; too large degrades to FCFS. Here quantum = 2.

Gantt: P1[0-2] P2[2-4] P1[4-6] P3[6-7] P2[7-9] P4[9-11] P1[11-13] P4[13-15] P1[15-16]

P	CT	TAT (CT-AT)	WT (TAT-BT)
P1	16	16	9
P2	9	7	3
P3	7	3	2
P4	15	10	6

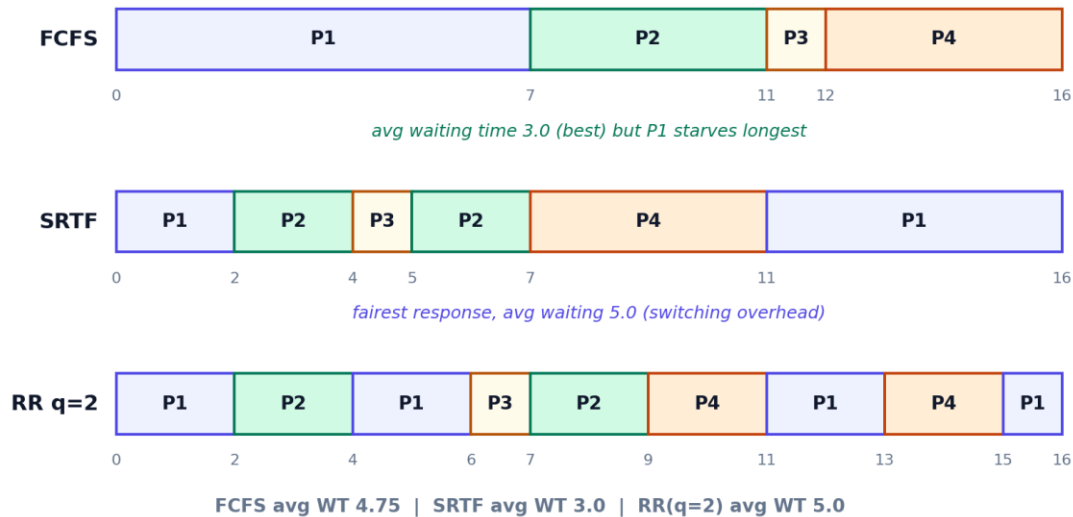
RR (q=2): Average TAT = $36/4 = 9.0$ Average WT = $20/4 = 5.0$

💡 Aha - Round Robin ek bathroom many students wali situation hai jisme har banda fixed 2 minute use kar sakta hai - time khatam, line ke peeche jao. Koi atka nahi rehta, sabko jaldi

pehla mauka milta hai, par baar baar andar-bahar karna thoda time kha jaata hai.

Same 4 processes, 3 schedulers (P1:0/7 P2:2/4 P3:4/1 P4:5/4)

convoy: short jobs stuck behind P1



Same 4 jobs, 3 schedulers. SRTF wins on average waiting time (3.0). FCFS suffers the convoy, RR is fairest on response.

Priority scheduling + MLFQ

Priority scheduling runs the highest-priority Ready job first (can be preemptive or not). Its flaw is starvation: a low-priority job may wait forever behind higher-priority arrivals. The fix is aging - slowly raise a waiting job's priority so it eventually runs.

Multilevel Feedback Queue (MLFQ) uses several ready queues with different priorities. A new job starts high. If it uses its full quantum (CPU-bound) it drops to a lower queue, and interactive jobs that block early stay high. It approximates SJF without knowing burst times. Tuning the queue counts and quanta is the deep part - not fresher-required.

💡 Aha - Priority scheduling VIP line hai - bade log pehle, aam aadmi peeche. Agar VIP aate rahe toh aam aadmi kabhi andar nahi ja payega (starvation). Aging woh rule hai jisme jitna zyada wait karoge utni tumhari priority apne aap badhti jaati hai.

Bonus: multicore, parallelism vs concurrency, affinity

Not core for a fresher round, but worth a one-line mention so a modern OS answer does not sound dated.

Concept	In one line
---------	-------------

Concept	In one line
Multicore CPU	Several independent cores on one chip; the OS schedules threads across all of them for true simultaneous execution
Concurrency	Many tasks make progress by interleaving - one core is enough (dealing with many things at once)
Parallelism	Tasks run at the literally same instant - needs multiple cores (doing many things at once)
CPU affinity	Pinning a thread to a specific core so it keeps its warm cache and avoids migration cost

💡 **Aha - Concurrency matlab ek bawarchi do dishes ke beech switch karke dono banata hai - lagta hai saath chal rahi hain. Parallelism matlab do bawarchi alag alag dish sach mein ek saath banate hain. Multicore = zyada bawarchi, aur affinity = ek bawarchi ko ek hi station pe rakhna taaki uska saaman paas rahe.**

@rathoreaditya

13. Beyond the basics (advanced, not all fresher-required)

Everything below goes deeper. Be honest about weight: synchronization, deadlock, and paging ARE fresher-tested - do not skip them. What you can defer for a first fresher round is the deepest internals: MLFQ tuning math, exact working-set computation, and file-system allocation details. Learn the rest. Treat those three as bonus depth.

14. Synchronization

When two threads touch the same shared data at the same time and the result depends on the timing, that is a race condition. The classic example: two threads both run $\text{counter} = \text{counter} + 1$. Each reads, increments, and writes back. If their reads interleave, one update is lost.

The section of code that touches shared data is the critical section. A correct solution must satisfy exactly three requirements:

6. Mutual exclusion - at most one process in the critical section at a time.
7. Progress - if no one is inside and someone wants in, the choice of who enters cannot be postponed forever.
8. Bounded waiting - there is a limit on how many times others can enter before a waiting process gets its turn (no starvation).

Peterson's solution is a software-only lock for two processes using a flag array and a turn variable. It is correct but only for two processes and assumes no instruction reordering, so it is theory, not what real kernels use.

The three real tools:

- Mutex (lock) - has ownership: only the thread that locked it may unlock it. Used for plain mutual exclusion.
- Semaphore - an integer with atomic `wait()` (decrement, block if negative) and `signal()` (increment). A binary semaphore is 0/1 and is a signaling mechanism any thread can signal. A counting semaphore takes any non-negative value and counts available units of a resource.
- Monitor - a higher-level construct (in languages like Java) that bundles shared data with the procedures that touch it, allowing only one thread inside at a time, using condition variables to wait and signal.

Semaphore - wait and signal

```
wait(S):          signal(S):
  S = S - 1        S = S + 1
  if (S < 0)        if (S <= 0)
    block(P)        wakeup(Q)
```

💡 **Aha - Semaphore hostel ke ek bathroom many students wala token system hai. Counting semaphore = teen bathroom hain toh token 3 se shuru, har banda andar jaate hi ek token**

kam. Token 0 ho gaya toh agla banda wait(). Binary semaphore = sirf ek bathroom, lock 0/1.

Mutex vs binary semaphore - the line interviewers love:

Aspect	Mutex	Binary semaphore
Ownership	Yes - only the locker can unlock	No owner - any thread can signal
Purpose	Mutual exclusion	Signaling between threads
Value	Locked / unlocked	0 or 1

Inter-process communication (IPC)

Processes have separate address spaces, so they cannot just share a variable like threads do. The OS gives them channels:

- Shared memory - both map the same RAM region. Fastest, but they must synchronize access themselves (with a semaphore or mutex).
- Message passing - the OS carries messages between them (pipes, message queues, sockets). Safer and simpler, but slower than shared memory.
- Pipe - a one-way byte stream, usually between a parent and child process.

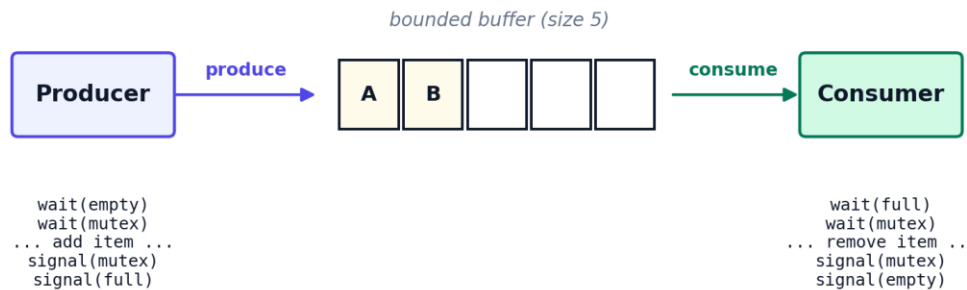
💡 **Aha - Shared memory matlab do log ek hi whiteboard pe likh-padh rahe hain - fast, par dono ek saath likhein toh gadbad, isliye lock chahiye. Message passing matlab ek doosre ko chit bhejna - thoda slow, par koi takraav nahi.**

15. Classic synchronization problems

Producer-Consumer (bounded buffer): a producer adds items, a consumer removes them, sharing a fixed-size buffer. Use three semaphores - empty (free slots), full (filled slots), and mutex (exclusive buffer access). The producer waits on empty and signals full. The consumer waits on full and signals empty.

Producer-Consumer: a shared buffer guarded by semaphores

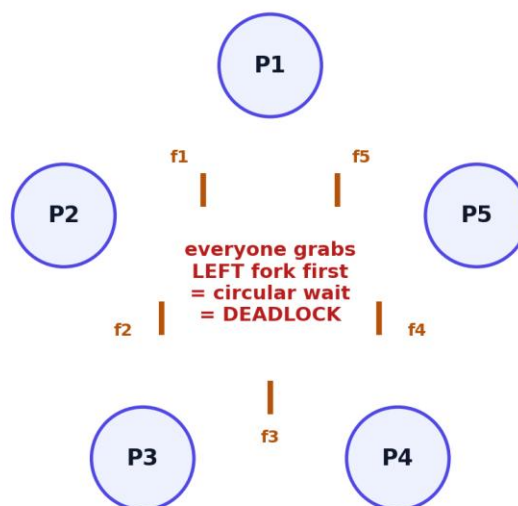
empty = free slots, full = filled slots, mutex = exclusive access. Producer waits when empty hits 0; consumer waits when full hits 0.



empty counts free slots, full counts filled slots, mutex guards the buffer. Each side waits when its count hits zero.

- Readers-Writers - many readers can read at once, but a writer needs exclusive access. The naive solution can starve writers, variants fix that.
- Dining Philosophers - five philosophers, five forks, each needs both neighboring forks to eat. If all grab the left fork first, everyone holds one and waits for the other - a deadlock.

Dining philosophers: 5 thinkers, 5 forks, the classic deadlock



Fix: make one philosopher pick the RIGHT fork first (break symmetry), or allow at most 4 to sit at once, or grab both forks atomically.

All-grab-left-first = circular wait = deadlock. Break the symmetry to fix it.

💡 Aha - Dining philosophers exactly woh scene hai jab table pe 5 log baithe hain aur har kisi ko khaane ke liye dono taraf ke spoon chahiye. Agar sab ek saath left spoon utha lein, toh

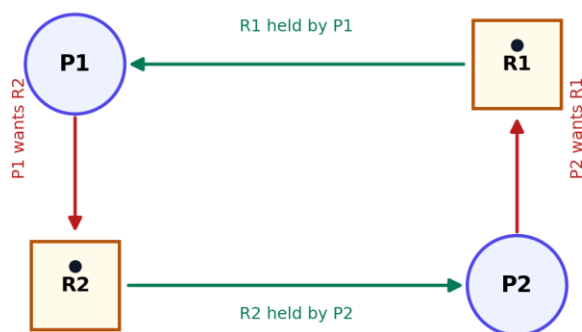
sabke paas ek-ek spoon hai aur koi doosra de hi nahi raha - sab bhookhe atak gaye.

16. Deadlock

Deadlock is when a set of processes are each waiting for a resource held by another in the set, so none can ever proceed. It needs ALL FOUR of the Coffman conditions to hold at the same time:

9. Mutual exclusion - at least one resource is non-shareable (one user at a time).
10. Hold and wait - a process holds at least one resource while requesting another.
11. No preemption - a resource is released only voluntarily by the holder.
12. Circular wait - a cycle of processes each waiting for the next one's resource.

Resource-allocation graph: a cycle = deadlock (single instances)



P1 holds R1 and waits for R2; P2 holds R2 and waits for R1. The cycle $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$ is the circular wait. Neither can move.

Resource-allocation graph. The cycle $P1 \rightarrow R2 \rightarrow P2 \rightarrow R1 \rightarrow P1$ is the circular wait. With single-instance resources, a cycle means deadlock.

💡 **Aha - Deadlock woh hai jab do log ek tang darwaze pe ek doosre ko pehle nikalne ka wait kar rahe hain - na tum hatoge na main, dono atke. Chaar shartein ek saath honi chahiye. Koi ek bhi tod do toh deadlock nahi hoga.**

Four ways to handle deadlock:

- Prevention - design so one Coffman condition can never hold (e.g. impose a global ordering on resources to kill circular wait, or require all resources up front to kill hold-and-wait).
- Avoidance - allow the conditions but only grant a request if the system stays in a safe state. Banker's algorithm does this: it grants a request only if there exists some order in which all processes can still finish.
- Detection - let deadlock happen, then find it with a wait-for graph (cycle = deadlock for single-instance resources) and recover.
- Recovery - break the deadlock by terminating processes or preempting (rolling back) their resources.

Safe state: a state is safe if there is at least one sequence in which every process can get its maximum resources, run, and release - with no process left stuck. A safe state guarantees no deadlock. An unsafe state only means deadlock is possible, not certain.

💡 Aha - Banker's algorithm bank manager jaisa hai - loan tabhi deta hai jab use yakeen ho ki baaki sabko bhi paisa lautana mumkin rahega. Agar ek bhi request se aisa risk ban jaaye ki sab atak sakte hain, toh woh request abhi rok deta hai. Yahi safe state check hai.

Deadlock vs starvation vs livelock

Interviewers love this distinction. All three mean no useful work gets done, but for different reasons.

Problem	What is happening	Progress?
Deadlock	Each process holds a resource and waits in a cycle, all blocked	No one moves
Starvation	A process is runnable but always passed over (low priority keeps losing)	Some never move
Livelock	Processes keep reacting to each other and changing state, never blocked	Everyone moves but nobody finishes

💡 Aha - Deadlock matlab dono gaadiyan jam mein khadi, koi nahi hil raha. Starvation matlab tumhari lane kabhi green nahi hoti, doosre nikalte rehte hain. Livelock matlab corridor mein do log ek doosre ko rasta dene ke liye baar baar same side hat-te rehte hain - hil rahe hain par koi aage nahi badhta.

17. Memory management

The OS has to fit many processes into limited RAM. Contiguous allocation gives each process one continuous block - simple, but it creates fragmentation:

Fragmentation	What it is	Where
Internal	Allocated block is bigger than needed, the unused tail is wasted inside the block	Fixed-size partitions, paging
External	Enough total free memory, but it is split into scattered small holes, none big enough	Variable-size partitions, segmentation

💡 Aha - Internal fragmentation tab hai jab tumhe 2 seat chahiye par poora 6-seater table mil gaya - baaki 4 seat khaali par lock. External tab hai jab total 5 seat khaali hain par alag alag tables pe bikhri hain, ek saath 3 doston ko kahin nahi bitha sakte.

Where to place a process: First / Best / Worst fit

With variable-size holes, which free hole do you give a new process? Three standard strategies interviewers ask by name:

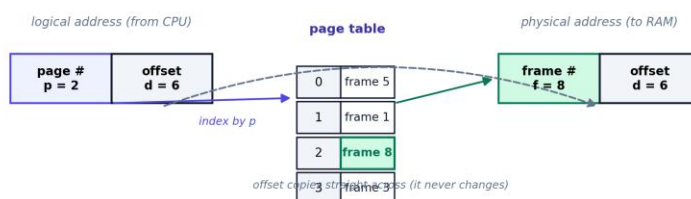
Strategy	Picks	Trade-off
First fit	The first hole big enough, scanning from the start	Fastest to decide
Best fit	The smallest hole that still fits	Least leftover, but leaves tiny unusable holes
Worst fit	The largest hole	Keeps big holes usable, wastes large space

💡 Aha - First fit matlab jo pehli khaali table dikhi usi pe baith gaye. Best fit matlab poora restaurant dekh ke sabse choti table jo fit ho jaaye usi pe - jagah bachti hai par chhoti chhoti useless tables reh jaati hain. Worst fit matlab hamesha sabse badi table lo.

Paging

Paging splits logical memory into fixed-size pages and physical memory into same-size frames. A page table maps each page number to a frame number. A logical address splits into a page number (index into the table) and an offset (copied straight through). Paging kills external fragmentation but can leave a little internal fragmentation in the last page.

Paging: logical address -> page table -> physical address



Physical address = frame_base + offset. The page table turns the page number into a frame number; the offset is untouched. The TLB caches recent page->frame lookups so most translations skip the table.

Logical address = (page, offset). The page table turns page into frame, the offset never changes. The TLB caches recent lookups.

💡 Aha - Paging cloud storage jaisa hai - tumhari file (process) chhote chhote fixed chunks (pages) mein toot jaati hai aur RAM ke kisi bhi khaali locker (frame) mein rakhi ja sakti hai. Page table woh index hai jo batata hai kaunsa chunk kis locker mein pada hai.

Segmentation

Segmentation splits memory by logical units - code, stack, heap - of variable size, matching how a programmer thinks. An address is (segment number, offset). It gives a natural view but brings back external fragmentation. Paging vs segmentation is a classic compare (see the tables section).

Virtual memory + demand paging

Virtual memory lets a process use more address space than physical RAM by keeping only the needed pages in memory and the rest on disk. Demand paging loads a page only when it is first accessed. If the accessed page is not in memory, the hardware raises a page fault. The OS finds the page on disk, loads it into a free frame (or evicts one via a page-replacement algorithm), updates the page table, and restarts the instruction.

💡 Aha - Virtual memory aise hai jaise tum poori library ghar nahi laate - jo kitaab abhi chahiye wahi shelf (RAM) pe rakhte ho, baaki godown (disk) mein. Page fault tab hota hai jab zaroori kitaab shelf pe nahi mili, toh use godown se mangana padta hai.

18. Page replacement + Belady's anomaly

When a page fault hits and all frames are full, a page-replacement algorithm chooses the victim to evict:

Algorithm	Evicts	Note
FIFO	The page loaded longest ago	Simple, can suffer Belady's anomaly
Optimal	The page used farthest in the future	Fewest faults possible, but needs the future - theoretical only
LRU	The page not used for the longest time	Good in practice, stack algorithm, no Belady
Clock	FIFO + a reference bit (second chance)	Cheap LRU approximation

Belady's anomaly: with FIFO, giving MORE frames can cause MORE page faults for some reference strings. It happens with FIFO only. LRU and Optimal are stack algorithms - adding frames can never increase their faults, so they never suffer Belady. Never claim LRU suffers Belady.

Belady's anomaly: FIFO faults can RISE with more frames

reference string: 3 2 1 0 3 2 4 3 2 1 0 4

FIFO, 3 frames

9 faults

more frames should help...

FIFO, 4 frames

10 faults (WORSE!)

...but FIFO does MORE work

LRU / Optimal

never gets worse with more frames

stack property: bigger cache always $\geq$ smaller cache

FIFO evicts the OLDEST-loaded page, even if it is hot. LRU and Optimal are stack algorithms, so adding a frame can never increase their faults. Belady's anomaly is a FIFO-only embarrassment.

For the string 3,2,1,0,3,2,4,3,2,1,0,4: FIFO gives 9 faults with 3 frames but 10 with 4 frames. LRU and Optimal never get worse with more frames.

💡 Aha - FIFO page replacement woh manager hai jo sabse purane employee ko nikaal deta hai chahe woh abhi sabse zyada kaam aa raha ho. Belady's anomaly uski bewakoofi hai - kabhi kabhi zyada frames dene par bhi woh aur zyada galat pages nikaal deta hai. LRU aur Optimal yeh galti kabhi nahi karte.

19. Thrashing, working set, TLB

- Thrashing - the system spends more time swapping pages in and out than doing useful work, because processes do not have enough frames. CPU utilization collapses.
- Working set - the set of pages a process is actively using in a recent time window. Give each process enough frames to hold its working set and thrashing drops. Exact working-set math is the deep, not-fresher part.
- TLB (Translation Lookaside Buffer) - a small fast cache of recent page-to-frame translations. A TLB hit skips the page-table lookup. A TLB miss costs an extra memory access.

💡 Aha - Thrashing tab hota hai jab exam se pehle tum itni saari kitaabein ek chhoti table pe rakhne ki koshish karte ho ki har naye chapter ke liye purani kitaab hatani padti hai, aur padhai kam, kitaab idhar-udhar karna zyada ho jaata hai. TLB woh sticky note hai jo bar-bar dekhe page ka location yaad rakhta hai.

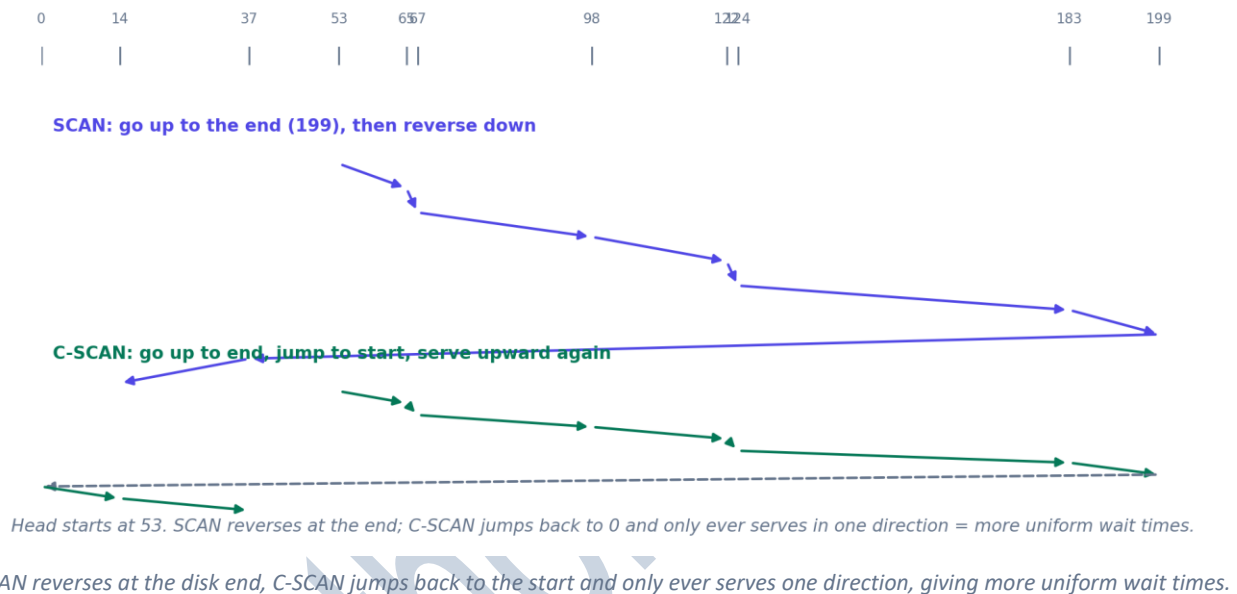
20. Disk scheduling

The disk arm must move across tracks to serve I/O requests. The algorithm decides the order, trying to minimize total head movement (seek time):

Algorithm	How it moves
FCFS	Serve requests in arrival order, can swing wildly across the disk

Algorithm	How it moves
SSTF	Serve the closest request next, better seek time but can starve far requests
SCAN (elevator)	Move in one direction serving all, reverse at the end
C-SCAN	Like SCAN but jumps back to the start instead of reversing - more uniform waits
LOOK	Like SCAN but reverses at the LAST request, not the disk end
C-LOOK	Like C-SCAN but jumps from the last request, not the disk end

Disk scheduling: SCAN (elevator) vs C-SCAN head path



💡 Aha - Disk scheduling building ki lift jaisa hai. SCAN wali lift upar tak jaati hai sabko leke, phir neeche aati hai. C-SCAN wali sirf upar jaate waqt logon ko leti hai, top pe pahunch ke seedha ground floor jump kar jaati hai - isse sabka wait barabar rehta hai.

21. File system allocation (brief)

- Contiguous - each file in one continuous block. Fast sequential access, suffers external fragmentation and hard to grow a file.
- Linked - each block points to the next. No external fragmentation, slow random access and a broken pointer loses the rest.
- Indexed - one index block lists all the file's block addresses. Good random access, the index block itself is overhead.

Allocation-method detail is the not-fresher-required depth flagged in §13. Know the three names and their one trade-off each.

22. Comparison tables (the ones interviewers love)

Scheduling algorithms

Algorithm	Preemptive	Starvation	Best for
FCFS	No	No	Simple batch, suffers convoy effect
SJF	No	Yes	Min average waiting (needs burst times)
SRTF	Yes	Yes	Lowest average waiting time
Round Robin	Yes	No	Time-sharing, good response time
Priority	Either	Yes (fix with aging)	Importance-based ordering
MLFQ	Yes	Possible	Mix of interactive + CPU-bound jobs

Process vs thread

Aspect	Process	Thread
Address space	Separate	Shared within the process
Switch cost	High	Low
Isolation	Strong (crash isolated)	Weak (shared fate)

Paging vs segmentation

Aspect	Paging	Segmentation
Block size	Fixed (pages / frames)	Variable (logical segments)
Fragmentation	Internal	External
View	Physical, OS-driven	Logical, programmer-driven
Address	(page, offset)	(segment, offset)

Internal vs external fragmentation

	Internal	External
Cause	Block bigger than the request	Free memory split into scattered holes
Waste is	Inside an allocated block	Between allocated blocks
Fix	Smaller block sizes	Compaction / paging

Mutex vs semaphore

Aspect	Mutex	Semaphore
Ownership	Yes (only locker unlocks)	No (any thread signals)
Values	Locked / unlocked	Counting (0..N) or binary (0/1)
Use	Mutual exclusion	Signaling + counting resources

Deadlock: prevention vs avoidance vs detection

Approach	Idea	Cost
Prevention	Break one Coffman condition by design	Low resource use / rigid
Avoidance	Grant only if state stays safe (Banker's)	Needs max-need known up front
Detection + recovery	Allow it, find it (wait-for graph), fix	Run detection + rollback cost

Spinlock vs mutex

Aspect	Spinlock	Mutex
Waiting thread	Busy-waits in a loop (keeps the CPU)	Sleeps, freeing the CPU
Context switch	None	Yes (block + wake)
Best for	Very short critical sections	Longer waits

Logical vs physical address

Aspect	Logical (virtual)	Physical
Seen by	The process / CPU	The actual RAM
Generated by	The CPU	The MMU after translation
Mapped via	Page table	It is the real location

Multiprogramming vs multitasking vs multithreading

Term	In one line
Multiprogramming	Many processes in memory, the CPU switches when one waits, to stay busy
Multitasking	Time-sharing so many tasks feel simultaneous on one CPU (rapid switching)
Multithreading	One process split into multiple threads sharing its memory

CPU-bound vs I/O-bound process

Type	Spends most time	Likes
CPU-bound	Computing (long bursts)	SJF / SRTF
I/O-bound	Waiting on I/O (short bursts)	Round Robin (good response)

23. Interview triggers - if they ask X, say Y

If they ask / say	You say
Difference between process and thread	Threads share the address space and are cheap to switch, processes are isolated and expensive
Which scheduling gives min waiting time	SJF / SRTF, but it needs burst times so it is a theoretical optimum
Can Round Robin starve a process	No - every process gets a slice each cycle
When does Belady's anomaly happen	FIFO only. LRU and Optimal are stack algorithms and never suffer it
What makes a deadlock	All four Coffman conditions at once, break any one to prevent it
Mutex vs binary semaphore	Mutex has ownership (only the locker unlocks). A binary semaphore is signaling, any thread can signal
Why is my multithreaded counter wrong	Race condition on a shared variable, guard the critical section with a lock
What is thrashing	Too little memory per process, so the CPU spends its time swapping pages, not computing
Difference between paging and segmentation	Paging is fixed-size and physical (internal frag). Segmentation is variable and logical (external frag)

24. Common mistakes + traps

- Saying Waiting Time = Completion - Arrival. That is Turnaround. Waiting = Turnaround - Burst.
- Claiming LRU suffers Belady's anomaly. It does not - only FIFO does.
- Saying deadlock needs any one Coffman condition. It needs all four at once.
- Calling a mutex and a binary semaphore the same thing. The mutex has ownership. The semaphore does not own anything.
- Saying a Running process that asks for I/O goes back to Ready. It goes to Waiting, then to Ready when I/O completes.
- Treating SJF as implementable directly. You cannot know future burst times. It is estimated or approximated by MLFQ.
- Forgetting the offset is copied unchanged in address translation - only the page number is translated.
- Assuming more frames always means fewer page faults. Belady's anomaly breaks that for FIFO.

25. Model interview answers - say it like this

The 50-question bank below is for rapid recall. This section is the opposite: the 17 questions an OS round actually opens with, each with a full answer you can speak in 20 to 40 seconds. Every answer follows the same shape - define it in one line, give one concrete example, then handle the follow-up before they ask. Read these out loud once; that is what makes them stick.

How to use a model answer: lead with the one-line definition so the interviewer knows you know it, then ground it in ONE example, then stop. Rambling past the example is how strong answers turn average. The follow-up line is the question they ask next - have it ready and you look two steps ahead.

Q. What is an operating system and what does it actually do?

Say this: An OS is the software layer between the hardware and your programs. It manages the CPU, memory, storage, and I/O devices, and it hands those resources out to processes fairly and safely. Concretely it does process and CPU scheduling, memory management, a file system, device drivers, and it isolates one program from another so a crash in one does not take down the machine. The core part that runs in privileged mode is the kernel.

Follow-up they hit you with: *Kernel vs OS? The kernel is the privileged core that touches hardware directly; the OS is the kernel plus the user-space utilities and shell around it.*

Q. Process vs thread - the real difference?

Say this: A process is a program in execution with its own private address space, so its memory is isolated from other processes. A thread is a unit of execution inside a process, and all threads of one process share that process's code, heap, and data, but each thread keeps its own stack, registers, and program counter. So threads are cheaper to create and switch between, and they can talk through shared memory directly, while processes need explicit inter-process communication.

Weak answer (skip it): *Saying threads have their own memory like processes. They share the heap; only the stack and registers are per-thread.*

Follow-up they hit you with: *Why is a thread switch cheaper than a process switch? Threads of the same process share the address space, so the OS does not flush the TLB or swap page tables - it only swaps registers and the stack pointer.*

Q. What happens in a context switch and why is it expensive?

Say this: A context switch is the OS saving the current process's state into its PCB - program counter, registers, and memory-management info - and loading the next process's state so it can resume. It is expensive because it is pure overhead: no user work happens during it, and on a process switch the CPU also flushes the TLB and the caches go cold, so the new process starts with cache misses. That is why too small a Round-Robin quantum kills throughput.

Follow-up they hit you with: *What triggers one? A timer interrupt at the end of a quantum, a process blocking on I/O, a higher-priority process becoming ready, or a system call that puts the process to sleep.*

Q. Concurrency vs parallelism?

Say this: Concurrency is dealing with many tasks at once by interleaving them - one CPU can switch between tasks so fast they all appear to progress together, even though only one runs at any instant. Parallelism is actually doing many tasks at the same instant, which needs multiple cores or CPUs. So concurrency is about structure, parallelism is about execution. You can have concurrency on a single core, but parallelism needs real hardware to run things side by side.

Follow-up they hit you with: A classic line: concurrency is two queues and one coffee machine; parallelism is two queues and two coffee machines.

Q. Preemptive vs non-preemptive scheduling?

Say this: In non-preemptive scheduling, once a process gets the CPU it keeps it until it finishes or blocks on its own, like FCFS or plain SJF. In preemptive scheduling, the OS can take the CPU back mid-execution, usually on a timer interrupt or when a higher-priority process arrives, like Round Robin or SRTF. Preemption gives better responsiveness and prevents one process from hogging the CPU, but it costs more context switches.

Follow-up they hit you with: Which one needs context switches more often? Preemptive - every quantum expiry or priority change can force a switch.

Q. Which scheduling algorithm would you pick and why?

Say this: It depends on the goal. FCFS is simple but suffers the convoy effect, where short jobs wait behind one long job. SJF or SRTF gives the minimum average waiting time, but it needs to know burst times in advance, so it is more a theoretical optimum. Round Robin is the practical choice for interactive systems because it is fair and starvation-free, with the time quantum tuning the trade-off between responsiveness and switch overhead. Priority scheduling fits when jobs really differ in importance, but it can starve low-priority jobs, which you fix with aging.

Follow-up they hit you with: How does MLFQ improve on these? It uses several priority queues and moves a process down when it uses too much CPU, so it approximates SJF without ever knowing the burst time up front.

Q. What is a race condition and how do you prevent it?

Say this: A race condition is when two or more threads access shared data at the same time and the final result depends on the exact timing of their operations. The classic case is two threads doing `count++` on the same variable: that increment is really read, add, write, and if the reads interleave, one update is lost. You prevent it by protecting the critical section so only one thread is inside at a time, using a mutex, a semaphore, or a monitor.

Follow-up they hit you with: What three properties must a correct critical-section solution have? Mutual exclusion, progress, and bounded waiting.

Q. Mutex vs semaphore - when do you use which?

Say this: A mutex is a lock for mutual exclusion: only one thread holds it at a time, and crucially only the thread that locked it can unlock it, so it has ownership. A semaphore is a signaling mechanism: it is a counter, and any thread can signal (increment) or wait (decrement) on it. Use a mutex to protect a single shared resource; use a counting semaphore to manage a pool of N identical resources, or to signal between threads, like producer-consumer.

Weak answer (skip it): Saying a mutex and a binary semaphore are the same thing. They behave alike for locking, but a binary semaphore has no ownership, so any thread can release it - that is the difference they want.

Follow-up they hit you with: Give a semaphore example: in producer-consumer, an empty semaphore counts free slots and a full semaphore counts filled slots, with a mutex guarding the buffer itself.

Q. Explain deadlock and the conditions for it.

Say this: Deadlock is when a set of processes are all blocked forever, each waiting for a resource that another one in the set is holding. It needs four conditions to hold at the same time, the Coffman conditions: mutual exclusion, where a resource is held by one process; hold and wait, where a process holds one resource and waits for another; no preemption, where you cannot forcibly take a resource away; and circular wait, where the waiting forms a cycle. Break any one of these and deadlock cannot happen.

Follow-up they hit you with: How do you handle deadlock in practice? Four strategies: prevention (break a Coffman condition), avoidance (Banker's algorithm keeps the system in a safe state), detection and recovery, or the ostrich approach - ignore it, which is what most general-purpose OSes actually do.

Q. Deadlock vs starvation?

Say this: Deadlock is a circular wait where a group of processes are permanently blocked and none can ever proceed, because each holds what another needs. Starvation is different: the process is not blocked in a cycle, it is just perpetually passed over - for example a low-priority job that keeps getting overtaken by higher-priority arrivals. In deadlock nobody makes progress; in starvation the system runs fine, but one unlucky process never gets its turn. Aging fixes starvation by slowly raising a waiting job's priority.

Follow-up they hit you with: *Is livelock the same as deadlock? No - in livelock the processes keep changing state in response to each other but still make no real progress, like two people stepping aside in the same direction repeatedly.*

Q. Paging vs segmentation?

Say this: Both map logical addresses to physical memory, but differently. Paging splits memory into fixed-size pages and frames, so it removes external fragmentation but causes a little internal fragmentation in the last page, and the address is just a page number plus offset with no real meaning to the programmer. Segmentation splits memory into variable-size, logical units like code, stack, and heap, which matches how a programmer thinks, but it brings back external fragmentation. Real systems often combine them as paged segmentation.

Follow-up they hit you with: *Which causes which fragmentation? Paging - internal; segmentation - external. That swap is the single most asked follow-up here.*

Q. What is virtual memory and how does demand paging work?

Say this: Virtual memory is the illusion that each process has a large, contiguous address space even when physical RAM is smaller, by keeping only the pages it currently needs in memory and the rest on disk. Demand paging is how it loads them: a page is brought in only when it is first accessed, not all up front. If a process touches a page that is not in memory, the hardware raises a page fault, the OS finds the page on disk, loads it into a free frame, possibly evicting another page, and restarts the instruction.

Follow-up they hit you with: *What is the TLB's role? It caches recent page-to-frame translations, so a TLB hit skips the page-table walk entirely - that is what keeps virtual memory fast.*

Q. FIFO vs LRU page replacement, and what is Belady's anomaly?

Say this: When memory is full and a new page is needed, the OS evicts one. FIFO evicts the oldest-loaded page, which is simple but can throw out a page that is still hot. LRU evicts the page that has not been used for the longest time, which tracks actual usage and performs much better. Belady's anomaly is the surprise that with FIFO, giving the process MORE frames can sometimes cause MORE page faults. LRU and Optimal never suffer it because they are stack algorithms.

Weak answer (skip it): *Claiming more frames always means fewer faults. That is exactly the intuition Belady's anomaly breaks, and only for FIFO.*

Follow-up they hit you with: *Why is Optimal not used in practice? It evicts the page that will be used farthest in the future, which needs knowledge of the future, so it only serves as a theoretical lower bound.*

Q. What is thrashing and how do you fix it?

Say this: Thrashing is when a system spends more time swapping pages in and out of disk than doing useful work. It happens when too many processes are running and each gets too few frames, so every process keeps page-faulting, which raises disk traffic, which drops CPU utilisation, which tempts the scheduler to admit even more processes - a vicious spiral. You fix it by reducing the degree of multiprogramming and by using the working-set model to give each process enough frames for the pages it is actively using.

Follow-up they hit you with: *What is the working set? The set of pages a process has referenced in the recent time window; keep that resident and page faults drop sharply.*

Q. What does fork() do, and what are zombie and orphan processes?

Say this: `fork()` creates a new child process by duplicating the calling process, so one call returns twice: it returns 0 to the child and the child's PID to the parent. A zombie is a child that has finished but whose parent has not yet called `wait()` to read its exit status, so its entry lingers in the process table. An orphan is the reverse: a child whose parent died first, and it gets adopted by `init`, process ID 1, which later reaps it.

Follow-up they hit you with: How do you avoid zombies? The parent should call `wait()` or `waitpid()` to collect the child's exit status, which clears the table entry.

Q. User mode vs kernel mode, and what is a system call?

Say this: Modern CPUs run in two privilege levels. User mode is restricted: an app cannot touch hardware or other processes' memory directly. Kernel mode is privileged: the OS runs here with full hardware access. A system call is the controlled doorway between them - when a program needs a privileged service like reading a file or creating a process, it makes a system call, which traps into the kernel, switches to kernel mode, does the work, and returns to user mode.

Follow-up they hit you with: Why have two modes at all? Isolation and safety: a buggy or malicious user program cannot crash the machine or read another process's memory because it never runs with hardware privileges.

@rathoreaditiya

26. Recently asked in interviews - what current cycles hit

These are the questions that keep showing up in 2025-26 placement and off-campus rounds, grouped by where they land. None of it is new theory, it is the same syllabus asked the way real interviewers ask it. If a row feels shaky, go back to that section before your round.

Where it shows up	What they actually ask
Process vs thread	Process vs thread and when you would pick a thread. What happens on a context switch and why it costs so much.
CPU scheduling	Which scheduler for a low-latency UI vs a batch server. Run SRTF and RR on this Gantt chart. Why can SJF starve a job.
Synchronization	Mutex vs semaphore, and where a binary semaphore differs from a mutex. Write producer-consumer with a semaphore. Show me a race condition.
Deadlock	State the four conditions. Is this resource-allocation graph deadlocked. How does Banker's decide a safe state.
Memory + paging	Paging vs segmentation. What virtual memory and demand paging really do. Trace a TLB miss into a page fault.
Page replacement	FIFO vs LRU on this reference string, count the faults. What is Belady's anomaly and which algorithms dodge it.
Linux + practical	What <code>fork()</code> returns in parent vs child. Zombie vs orphan. User mode vs kernel mode and the path a system call takes.

💡 **Aha - Pattern dekho:** har round ghoom-phir ke wahi saat areas pe aata hai. In saat rows ko bol ke practice karo, baaki sheet revision ban jaati hai.

OS concurrency also shows up as a coding round

Product companies increasingly test synchronization as a thread-coding problem, not just theory. These six on LeetCode are the same concepts from sections 14 and 15, written as code you have to make thread-safe.

- [1114. Print In Order](#) - force three threads to run in order (condition variable)
- [1115. Print FooBar Alternately](#) - alternate two threads with clean signaling
- [1116. Print Zero Even Odd](#) - three threads, one shared turn (semaphores)
- [1117. Building H2O](#) - a barrier: assemble 2 H + 1 O before releasing
- [1195. Fizz Buzz Multithreaded](#) - four threads, one counter, coordinate the prints
- [1226. The Dining Philosophers](#) - the classic deadlock-avoidance problem in code

Where these come from: [GFG OS interview questions](#) and the [OS last-minute notes](#), cross-checked against recent off-campus experiences.

27. Interview FAQ bank - 50 questions

Quick-recall once the model answers above are solid. One or two lines each; if you can answer all 50 fast, your OS fundamentals are ready.

Question	Answer
What is an operating system?	Software that manages hardware and provides services to programs: process, memory, file, device, and I/O management.
Kernel vs OS?	The kernel is the core that runs in privileged mode and manages CPU, memory, and devices. The OS is the kernel plus user-space utilities.
Process vs program?	A program is passive code on disk. A process is a program in execution with its own memory, registers, and state.
Process vs thread?	A process has its own address space. Threads inside a process share that space but keep their own stack and registers.
What is in a PCB?	PID, state, program counter, CPU registers, scheduling and memory info, accounting, and I/O status for one process.
Name the process states.	New, Ready, Running, Waiting (Blocked), Terminated.
Does a Running process go to Ready or Waiting on an I/O request?	Waiting. It returns to Ready only after the I/O completes.
What is a context switch?	Saving one process's state to its PCB and loading another's. It is pure overhead with no useful work done.
What does fork() return?	0 to the child and the child's PID to the parent, so one call returns twice.
What is a zombie process?	A finished child whose parent has not yet called wait() to read its exit status, so its PCB lingers.
What is an orphan process?	A process whose parent died first. It is adopted by init (PID 1), which later reaps it.
User mode vs kernel mode?	User mode runs apps with limited privileges. Kernel mode runs the OS with full hardware access. System calls cross between them.
What is a system call?	A controlled request from a user program into the kernel for a privileged service, via a trap that switches to kernel mode.
Name the thread models.	Many-to-one, one-to-one, and many-to-many mappings of user threads to kernel threads.
Turnaround time formula?	Turnaround = Completion Time - Arrival Time.
Waiting time formula?	Waiting = Turnaround Time - Burst Time.
Response time formula?	Response = first time on the CPU - Arrival Time.
Which scheduling minimizes average waiting time?	SJF (and its preemptive form SRTF), but it needs burst times, so it is a theoretical optimum.
What is the convoy effect?	In FCFS, short jobs stuck behind one long job all wait, inflating average waiting time.
Can FCFS cause starvation?	No - every job eventually runs since order is purely by arrival.
Can SJF cause starvation?	Yes - a long job can keep getting passed over by shorter arrivals.
Can Round Robin cause starvation?	No - each Ready process gets a time slice every cycle.
What happens if the RR quantum is too small or too large?	Too small wastes time on context switches. Too large degrades RR into FCFS.
What is aging in scheduling?	Gradually raising a waiting process's priority so it eventually runs, preventing starvation.
What is MLFQ?	Multilevel feedback queue: multiple priority queues, jobs move between

	them based on CPU usage, approximating SJF without burst times.
What is a race condition?	When the result of concurrent access to shared data depends on the timing of the threads.
What is the critical section?	The part of code that accesses shared data and must run with mutual exclusion.
Three critical-section requirements?	Mutual exclusion, progress, and bounded waiting.
Mutex vs binary semaphore?	A mutex has ownership (only the locker unlocks). A binary semaphore is a signaling mechanism any thread can signal.
Counting vs binary semaphore?	A counting semaphore takes any non-negative value (counts resource units). A binary semaphore is restricted to 0 or 1.
What is a monitor?	A high-level construct that bundles shared data with its procedures, allowing one thread inside at a time, using condition variables.
What is Peterson's solution?	A software-only mutual-exclusion algorithm for two processes using a flag array and a turn variable.
Producer-Consumer semaphores?	empty (free slots), full (filled slots), and mutex (exclusive buffer access).
Why does Dining Philosophers deadlock?	If all philosophers grab the left fork first, each holds one and waits for the other - a circular wait.
The four Coffman conditions?	Mutual exclusion, hold and wait, no preemption, and circular wait - all four at once.
How does deadlock prevention work?	Design the system so one Coffman condition can never hold (e.g. order resources to kill circular wait).
What is the Banker's algorithm?	A deadlock-avoidance algorithm that grants a request only if the system stays in a safe state.
What is a safe state?	A state where some sequence lets every process get its max resources, finish, and release - so no deadlock occurs.
How is deadlock detected?	With a wait-for graph. A cycle means deadlock for single-instance resources.
Internal vs external fragmentation?	Internal: wasted space inside an allocated block. External: free memory scattered in holes too small to use.
What is paging?	Splitting memory into fixed-size pages and frames, mapped by a page table. It removes external fragmentation.
How does address translation work?	The logical address splits into page number and offset. The page table maps page to frame, and the offset is unchanged.
What is the TLB?	A small cache of recent page-to-frame translations. A hit skips the page-table lookup.
What is a page fault?	An access to a page not currently in memory. The OS loads it from disk, possibly evicting another page.
What is demand paging?	Loading a page only when it is first accessed, rather than all up front.
Name the page-replacement algorithms.	FIFO, Optimal, LRU, and Clock (second-chance).
When does Belady's anomaly occur?	With FIFO only: more frames can cause more faults. LRU and Optimal are stack algorithms and never suffer it.
Why is Optimal page replacement not used?	It evicts the page used farthest in the future, which requires knowing the future, so it is only a theoretical baseline.
What is thrashing?	When processes lack enough frames and the CPU spends its time swapping pages instead of computing.
Name disk-scheduling algorithms.	FCFS, SSTF, SCAN (elevator), C-SCAN, LOOK, and C-LOOK.

28. The 30-minute revision card

The formulas:

Turnaround = Completion - Arrival. Waiting = Turnaround - Burst. Response = first CPU - Arrival.

One-liners to fire back:

- Process = isolated, own memory. Thread = shared memory, cheap switch.
- Running asks for I/O -> Waiting (not Ready). I/O done -> Ready.
- FCFS = convoy effect, no starvation. SJF/SRTF = min waiting, can starve.
- Round Robin = fair, no starvation, quantum sets the trade-off.
- Priority can starve -> fix with aging. MLFQ approximates SJF.
- Critical section needs: mutual exclusion, progress, bounded waiting.
- Mutex = ownership. Binary semaphore = signaling. Counting = resource units.
- Deadlock = all 4 Coffman conditions at once: mutual exclusion, hold and wait, no preemption, circular wait.
- Banker's algorithm = grant only if the state stays safe.
- Address translation: page -> frame via page table, offset unchanged. TLB caches it.
- Page replacement: FIFO, Optimal (theory), LRU, Clock.
- Belady's anomaly = FIFO only. LRU + Optimal never suffer it.
- Thrashing = too few frames, fix by sizing the working set.
- Disk: SCAN reverses at the end, C-SCAN jumps back to the start.

If interviewer says X, you say Y:

- "Min waiting time" -> SJF / SRTF (theoretical, needs burst times).
- "More frames, more faults" -> Belady's anomaly, FIFO only.
- "Two threads, wrong total" -> race condition, lock the critical section.

Two traps that cost you easy points:

- Mixing up waiting and turnaround time in a Gantt question - recompute, do not guess.
- Saying LRU suffers Belady - it never does.

29. Final word

You have the formulas, the four Gantt algorithms worked end to end, the deadlock conditions, and the page-replacement rules. An OS round often includes one Gantt chart, one deadlock question, and one memory question. You have already practised all three. Draw it calmly and you will be ready for the vast majority of OS interview questions.